Urban Traffic Twin

Jumeau numérique d'un carrefour à feux parisien dans SUMO, calibré sur données réelles

Bd de l'Hôpital × Bd Saint-Marcel — Pitié-Salpêtrière, Paris 13e

Dossier de validation : explication complète du projet et localisation exacte du code de chaque simulation

Élément	Valeur
Date du document	30 octobre 2026
Version du code analysée	commit `f58409b` (branche `interface-v2`, identique à `origin/main`)
Dépôt	github.com/motezjerbi/urban-traffic-twin
Langage / outils	Python 3, Eclipse SUMO + TraCI/libsumo, Gymnasium + Stable-Baselines3, XGBoost, FastAPI, Streamlit, DuckDB, Supabase (miroir optionnel)

Comment lire ce document

Chaque fonctionnalité est expliquée puis **localisée dans le code** sous la forme `chemin/fichier.py:ligne` (chemins relatifs à la racine du dépôt ; le code Python est dans `src/traffic_twin/`). Les encadrés gris sont des **extraits réels** du code source, lus automatiquement dans les fichiers au moment de la génération du PDF, avec leurs vrais numéros de ligne : ce que vous voyez ici est exactement ce que le professeur verra en ouvrant le fichier.

Le chapitre 7 (**l'ambulance**) suit pas à pas un véhicule d'urgence depuis sa création jusqu'à la mesure de son temps de parcours, et explique trois façons de la montrer en direct. L'annexe A est un index « je cherche X → fichier:ligne » pour répondre vite pendant la soutenance. Le chapitre 21 prépare les questions probables du jury.

Sommaire

1. Le projet en bref **5**

Les contrôleurs de feux comparés . . . 5

Le projet en 10 parties : état d'avancement . . . 5

2. Architecture et organisation du dépôt **6**

Flux de données en une phrase . . . 6

Stockage . . . 6

3. Une simulation de A à Z **7**

Les étapes et leur emplacement . . . 7

Le cœur : la boucle principale . . . 7

Paramètres d'un run Pitié (configs/pitie_fixed.yaml) . . . 8

Commandes pour lancer une simulation . . . 8

Fichiers produits par un run . . . 8

4. Le réseau SUMO et le programme de feux **9**

Construction du réseau à partir d'OpenStreetMap . . . 9

Les 17 phases (sumo/network/pitie/pitie.tll.xml:51-69) . . . 9

5. La demande : qui circule dans la simulation **10**

Voitures (calibrées sur les capteurs) . . . 10

Bus (configs/pitie/demand.yaml:19-24, horaires supposés) . . . 10

Piétons . . . 10

Ambulance . . . 10

6. Les contrôleurs de feux **11**

Le contrat commun . . . 11

S0 – Feux fixes . . . 11

S1 – Actuated (détection de lacunes) . . . 11

S2 – Contrôleur par règles (solution principale) . . . 11

S3 – Apprentissage par renforcement . . . 12

S4 et les variantes « nopreempt » . . . 13

7. L'ambulance : de sa création à son temps de parcours **14**

7.1 La chaîne d'appels complète (un pas de simulation) . . . 14

7.2 Création de l'ambulance . . . 14

7.3 Détection en amont . . . 14

7.4 Gestion de la demande de préemption . . . 15
7.5 Comment la phase verte est imposée . . . 15
7.6 Retour à la normale . . . 16
7.7 Mesure du temps de l'ambulance (KPI) . . . 16
7.8 Preuve statistique : critère A-05 . . . 17
7.9 Montrer l'ambulance en direct (3 façons) . . . 17
7.10 Tests qui couvrent l'ambulance . . . 19
8. La priorité bus (TSP) 20
9. Les piétons et le bouclier piéton 21
10. Le SafetyGuard : la sécurité avant tout 22
Audit indépendant après chaque run (A-04) . . . 22
11. Blocage du carrefour et incidents 23
JunctionMonitor (sim/junction.py:83) . . . 23
Incidents (sim/incidents.py) . . . 23
12. Les indicateurs (KPI) par type d'usager 24
13. Données, choix du carrefour, calibration et validation 25
Source et nettoyage . . . 25
Choix du carrefour (score 79,3 %) . . . 25
Calibration de la demande . . . 25
Validation simulation vs réalité (spec 13.4) . . . 25
14. Le modèle de temps de parcours (ETA) 27
Conseil de départ pour l'étudiant . . . 27
15. L'apprentissage par renforcement (S3) 28
L'environnement Gymnasium (rl/env.py:347 TrafficEnv) . . . 28
La récompense . . . 28
Entraînement et sélection . . . 28
Le verdict « FALLBACK » (spec 10.6), à bien expliquer . . . 28
16. Évaluation finale S0 à S4 30
Protocole (pré-enregistré le 2026-10-12) . . . 30
Tableau final (moyennes sur 10 graines) . . . 30
A-06 : le RL comparé aux références . . . 30
17. L'API et l'interface 32
API REST FastAPI (src/traffic_twin/api/main.py, create_app() ligne 919) . . . 32
Interface web src/traffic_twin/app/mock.html (servie par l'API) . . . 32
Tableau de bord Streamlit de l'opérateur (app.py) . . . 32
18. Lancer la démonstration (Windows) 34
Parcours de démonstration en 8 étapes (graine 4000) . . . 34
19. Critères d'acceptation 35
20. Les tests automatiques 36
21. Limites et questions probables du jury 37
Limites à assumer . . . 37

Questions / réponses 37
Annexe A – Index rapide « je cherche… » 38

1. Le projet en bref

Objectif. Construire le *jumeau numérique* d'**un** carrefour à feux réel de Paris dans le simulateur de trafic microscopique **Eclipse SUMO**, le calibrer sur les **capteurs permanents de la Ville de Paris** (débit horaire et taux d'occupation des boucles électromagnétiques), puis **comparer plusieurs stratégies de pilotage des feux** et montrer leur effet pour **cinq types d'usagers** : étudiant (voiture), ambulance, bus, piéton, gestionnaire (opérateur).

Le site. Carrefour Bd de l'Hôpital × Bd Saint-Marcel, à côté de l'hôpital **Pitié-Salpêtrière** (Paris 13e, nœud capteur 302). Il a été choisi par une grille multicritère (score 79,3 %, chapitre 13) parce qu'il cumule capteurs fiables, lignes de bus (91, 24, 57), passages piétons et un **couloir d'urgence vers l'hôpital**. Un carrefour « grille » provisoire (Partie 2) est conservé comme scénario de régression.

Les contrôleurs de feux comparés

Label	Nom	Principe	Fichier de config	Code
S0	Feux fixes	Cycle fixe de 123 s, verts fixes	`configs/pitie_fixed.yaml`	`control/fixed.py:29`
S1	Actuated	Détection de lacunes (gap-out) sur boucles amont	`configs/pitie_actuated.yaml`	`control/actuated.py:72`
S2	Règles (solution principale)	Actuated + préemption ambulance + priorité bus + bouclier piéton + anti-blocage	`configs/pitie_rules.yaml`	`control/rules.py:280`
S2_noP	Règles sans préemption	S2 avec la préemption ambulance désactivée	`configs/pitie_rules_nopreempt.yaml`	idem S2
S3	Apprentissage par renforcement	Agent DQN entraîné (modèle figé)	`configs/pitie_rl_s3.yaml`	`control/rl.py:280`
S4	RL sans préemption	Même politique que S3, préemption désactivée (hors distribution)	`configs/pitie_rl_s3_nopreempt.yaml`	idem S3

Les labels S0…S4 ne sont pas dans le code Python : ils associent un nom à un fichier de configuration dans `configs/evaluation.yaml:7-24`. Le registre Python ne connaît que les noms `fixed`, `actuated`, `rules`, `rl` (`src/traffic_twin/control/registry.py:41`).

Le projet en 10 parties : état d'avancement

Partie	Contenu	Statut
1	Dépôt, schéma Supabase, exploration des capteurs, choix du carrefour	fait
2	Réseau SUMO (feux, passages piétons, bus), demande provisoire, feux fixes, premiers KPI	fait
3	Réseau réel OSM, calibration de la demande, validation sim/réalité, actuated (S1)	fait (visite terrain en attente)
4	Contrôleur par règles S2, Safety Guard, préemption, priorité bus, KPI par usager	fait (visite terrain en attente)
5	Modèle ETA (XGBoost quantiles + intervalle conforme), API v1, écrans	fait
6	Environnement Gymnasium, contrôleur RL, premiers entraînements	fait
7	Entraînement RL complet, tuning, sensibilité aux poids, sélection de S3	fait (verdict FALLBACK)
8	Évaluation finale S0–S4 sur 10 graines, tests statistiques	fait (provisoire)
9	Application (vues usagers, replay), kit de tests utilisateurs	fait, tests utilisateurs en attente
10	Rapport, diaporama, outillage de démo, répétition automatique	fait, répétition humaine et soutenance en attente

Honnêteté scientifique, à dire au jury. Le programme de feux réel n'a pas encore été relevé sur place : il est marqué **ASSUMED** (supposé). Les comparaisons sont donc **provisoires** jusqu'à la visite terrain (protocole : `docs/field_protocol.md`). Les débits simulés, eux, sont validés contre les capteurs (A-02 PASS). Les tests avec 3 à 5 vrais utilisateurs (A-10) restent à faire.

2. Architecture et organisation du dépôt

Dossier	Rôle
`src/traffic_twin/data/`	Téléchargement Open Data Paris, nettoyage, calendrier (jours types), calibration de la demande, score des carrefours
`src/traffic_twin/sim/`	Cœur de simulation : réseau, demande, lancement SUMO, session pas-à-pas, détecteurs, plan de feux, KPI, audit de sécurité, ambulance, bus, incidents
`src/traffic_twin/control/`	Les contrôleurs (fixed, actuated, rules, rl), la préemption, le bouclier piéton, le SafetyGuard, le registre
`src/traffic_twin/rl/`	Environnement Gymnasium, observation, récompense, entraînement, campagne de tuning, étude de sensibilité
`src/traffic_twin/ml/`	Modèle de temps de parcours (ETA) : dataset, features, XGBoost, intervalle conforme, conseil de départ
`src/traffic_twin/validation/`	Comparaison simulation / réalité : débits (GEH), feux, files, boucle de calibration
`src/traffic_twin/evaluation/`	Protocole d'évaluation pré-enregistré, statistiques (Wilcoxon, Hodges-Lehmann, Holm), audits A-04/A-05/A-06
`src/traffic_twin/api/`	API REST FastAPI (lecture des résultats via DuckDB)
`src/traffic_twin/app/`	Interface web `mock.html` (vues usagers + replay 2D/3D), client
`app.py`	Tableau de bord Streamlit de l'opérateur (8 onglets)
`src/traffic_twin/demo/`, `report/`, `usertests/`	Outillage de démonstration, génération du rapport/diaporama, kit de tests utilisateurs
`configs/`	Tous les paramètres en YAML (réseau, demande, contrôle, règles, RL, évaluation)
`sumo/network/`	Fichiers SUMO : réseau Pitié (`pitie/*.xml`), OSM brut, grille
`tests/`	Tests automatiques pytest (unitaires + bout-en-bout dans SUMO)
`tasks.py`	Lanceur de tâches multiplateforme (équivalent du Makefile sous Windows)
`data/`	Résultats locaux (non versionné) ; `docs/demo/demo_pack.zip` permet de les restaurer

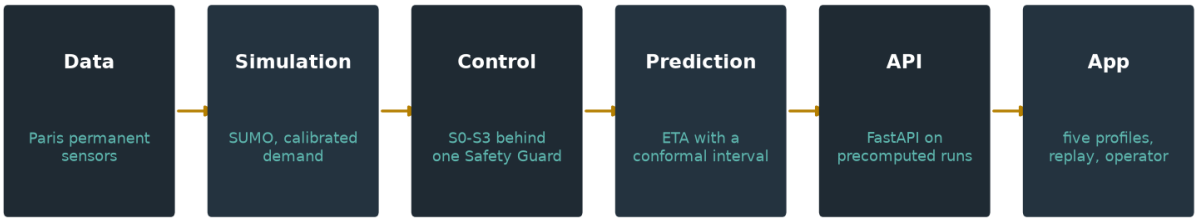


Figure 1 – Chaîne de traitement du projet (`docs/figures/report/pipeline.png`)

Flux de données en une phrase

Capteurs Paris → nettoyage → jours types → **débits calibrés par branche** → fichier de demande SUMO → **simulation** pilotée seconde par seconde par un **contrôleur** filtré par le **SafetyGuard** → fichiers de sortie (trajets, états, KPI) → **évaluation statistique** → **API** → **interface web / Streamlit**.

Stockage

Le stockage principal est **local** : chaque simulation écrit un dossier `data/results/<scenario>_seed<N>/`. L'API lit ces fichiers avec **DuckDB** (`src/traffic_twin/api/db.py:146`). **Supabase** (Postgres + bucket `sim-logs`) est un **miroir optionnel** : schéma dans `supabase/migrations/20261011000000_init_schema.sql` (7 tables, RLS activé), chargement par `python tasks.py db-load` (capteurs, `src/traffic_twin/db/load_sensors.py`) et `python tasks.py db-runs` (runs, `src/traffic_twin/db/load_run.py`). Aucun test n'a besoin de Supabase.

3. Une simulation de A à Z

Toutes les simulations (S0 à S4, avec ou sans ambulance) passent par **le même chemin de code**. Seuls le contrôleur et quelques blocs de configuration changent.

Les étapes et leur emplacement

#	Étape	Où dans le code
1	Ligne de commande, lecture du YAML (avec héritage `extends:`)	`sim/run_scenario.py:260` `main()` → `config.py:158` `load_scenario()`
2	Création du contrôleur demandé (`controller:` dans le YAML)	`sim/run_scenario.py:211` → `control/registry.py:41` `make_controller()`
3	Ouverture de la session de simulation	`sim/run_scenario.py:217` → `sim/session.py:96` `SimSession`
4	Génération de la demande (voitures, bus, piétons, ambulance) en XML SUMO	`sim/session.py:165` → `sim/demand.py:436` `write_demand()`
5	Écriture des détecteurs (boucles, files)	`sim/session.py:166` → `sim/detectors.py:298`
6	Lancement de SUMO (libsumo, ou TraCI + sumo-gui avec `--gui`)	`sim/session.py:163` `start()` → `sim/sumo.py:46` `sumo_command()`
7	Création du plan de feux, du SafetyGuard, des détecteurs ambulance/bus, de la préemption	`sim/session.py:180-213`
8	**Boucle principale** : observer → décider → appliquer	`sim/run_scenario.py:222-228`
9	Un pas : SafetyGuard → séquenceur de phases → `setPhase` → `simulationStep`	`sim/session.py:277-311` `step()`
10	Fin : demande terminée **et** réseau vide (ou +1800 s de purge)	`sim/session.py:235` `done`
11	Lecture des sorties SUMO et calcul des KPI par usager	`sim/results.py:60` → `sim/kpi.py:136` et `:217`
12	Audit de sécurité des phases réellement affichées	`sim/audit.py:65` `audit_phases()`
13	Écriture des fichiers de résultats	`sim/results.py` `write_results()`

Chemins relatifs à `src/traffic_twin/`.

Le cœur : la boucle principale

▸ **`src/traffic_twin/sim/run_scenario.py` : lignes 222–228** — *toute la simulation tient dans cette boucle*

```
222  with sim:
223      controller.bind(sim)
224      controller.reset(seed)
225      for o in observers:
226          o.bind(sim)
227      while not sim.done:
228          sim.step(controller.decide(sim.observe()))
```

À chaque seconde simulée : `sim.observe()` construit une **observation** (étape de feu courante, temps passé, files, lacunes des boucles, attente piétons, appels ambulance/bus) ; le contrôleur répond par **l'étape verte qu'il souhaite** ; `sim.step()` la fait passer par le SafetyGuard puis l'applique dans SUMO.

▸ **`src/traffic_twin/sim/session.py` : lignes 284–295** — *un pas : filtre de sécurité puis commande du feu*

```
284  state = dataclasses.replace(
285      self.plan.state(obs.t),
286      ped_wait=self._guard_waits,
287      preempt_target=obs.extras.get(EXTRA_PREEMPT),
288  )
289  requested = stage
290  stage = self.guard.apply(stage, state)
291  self.last_request = (int(requested), int(stage))
292  phase = self.plan.request(stage)
293  with self._alive():
294      self.traci.trafficlight.setPhase(self.tls, phase)
295      self.traci.trafficlight.setPhaseDuration(self.tls, 1e6)  # we drive switching
```

Point important : **c'est Python qui pilote chaque changement de feu**, pas SUMO (`setPhaseDuration(..., 1e6)` empêche SUMO de basculer seul). Les jaunes et rouges de dégagement

sont insérés automatiquement par `SignalPlan` (`sim/signal.py:84-113`) : un contrôleur ne peut jamais les raccourcir.

Paramètres d'un run Pitié (`configs/pitie_fixed.yaml`)

Paramètre	Valeur	Signification
`begin_s`	27000	début à 07:30
`duration_s`	7200	2 heures de demande (07:30–09:30)
`step_length_s`	1.0	pas de simulation 1 s
purge `DRAIN_S`	1800	temps max pour vider le réseau après la demande (`sim/sumo.py:12`)
`--time-to-teleport`	300	option SUMO anti-blocage
`--collision.check-junctions`	true	détection des collisions dans le carrefour
`state_log_every_s`	5	positions de tous les véhicules/piétons toutes les 5 s (replay)
graines	1–10 évaluation ; 101–103 calibration ; 4000 démo	`configs/evaluation.yaml:2`

Commandes pour lancer une simulation

Objectif	Commande (racine du dépôt, PowerShell)
Simulation sans interface (S0)	`.venv\Scripts\python.exe tasks.py sim --scenario configs/pitie_fixed.yaml --seeds 101`
Même chose dans **sumo-gui** (fenêtre graphique)	`.venv\Scripts\python.exe tasks.py sim-gui --scenario configs/pitie_rules.yaml --seeds 101`
Appel direct du module	`python -m traffic_twin.sim.run_scenario --config configs/pitie_rules.yaml --seed 101 --gui`
Vérification statique du programme de feux	`python tasks.py tls-check`

> Les graines **1 à 10 sont verrouillées** (« run lock ») car elles servent à l'évaluation officielle : `tasks.py` refuse de les relancer. Pour une démonstration, utiliser une autre graine (101, 4000…). Sans `--scenario`, `tasks.py sim` simule la grille de la Partie 2, pas Pitié.

Fichiers produits par un run

Fichier (dans `data/results/<scenario>_seed<N>/`)	Contenu
`tripinfo.xml`, `stops.xml`, `statistics.xml`	sorties brutes SUMO (trajets, arrêts bus, collisions, téléportations)
`kpi_run.csv`	KPI au format long : scenario, seed, user_type, kpi, value
`trip.parquet`	un enregistrement par véhicule et par piéton
`state_log.parquet`	positions x, y, vitesses, état des feux toutes les 5 s (alimente le replay)
`events.parquet`	événements : préemption, priorité bus, interventions du SafetyGuard
`detector.parquet`, `queue.parquet`, `signal.parquet`	boucles virtuelles, files, changements de feux
`run.json`	métadonnées : run_id, commit git, config fusionnée, hash, santé, audit sécurité, préemption

4. Le réseau SUMO et le programme de feux

Construction du réseau à partir d'OpenStreetMap

Le réseau réel est extrait d'OpenStreetMap (`sumo/network/osm/pitie.osm`) en deux étapes : découpage/filtrage (`sim/osm_import.py:78 osm_to_raw()`, centre 48.83992, 2.36172), puis « curation » en branches nommées `n_in`, `s_in`, `w_in`, `nb_in`, `wb_in`… autour d'un nœud central `C` (`sim/osm_import.py:170 curate()`). `netconvert` assemble ensuite `sumo/network/pitie/pitie.net.xml` (`sim/network.py:47`). Le réseau est **versionné** : une simulation ne le reconstruit pas.

- **5 branches** : N, S, W (voitures) et NB, WB (couloirs **bus/vélo/urgence** uniquement).
- **16 liens signalisés** : 12 véhiculaires + 4 passages piétons (`:C_c0` à `:C_c3`).
- **17 phases** regroupées en **5 étapes vertes** ; cycle de **123 s** (74 s de vert + 49 s de dégagement), `configs/pitie/network.yaml:44-50`.
- L'ambulance peut emprunter la voie bus de `s_in` (`configs/pitie/network.yaml:42`).

Les 17 phases (`sumo/network/pitie/pitie.tll.xml:51-69`)

Phase	Durée (s)	Rôle
0	36	**HOP** vert : N tout droit, S tout droit, piétons NB/WB/W
1–3	1 / 3 / 5	dégagement HOP (dont jaune 3 s)
4	6	**S_LEFT** vert : S tout droit + gauche, bus 57N, piétons WB
5–6	3 / 7	dégagement S_LEFT (jaune puis rouge intégral véhicules)
7	20	**STM** vert : branche W, N à droite, piétons NB/S/WB
8–9	3 / 4	dégagement STM
10	6	**BUS_91** vert : bus 91/24 dans les deux sens, N à droite, piétons S
11–13	1 / 3 / 8	dégagement BUS_91
14	6	**BUS_57** vert : bus 57S, bus 91/24 W, N à droite
15–16	3 / 8	dégagement BUS_57 (jaune puis rouge intégral)

Les temps de vert ci-dessus sont ceux du **feu fixe S0** (`configs/pitie/network.yaml:109` : HOP 36, S_LEFT 6, STM 20, BUS_91 6, BUS_57 6). Pour les autres contrôleurs, la durée des verts est décidée dynamiquement ; seules les phases de dégagement sont fixes.

Contrôle statique du programme (verts antagonistes, intervert, attente piéton maximale théorique ≤ 88 s) : `sim/tls_check.py:481 check_scenario()`.

5. La demande : qui circule dans la simulation

Tous les usagers sont générés dans un fichier `sumo/demand/<scenario>.rou.xml` par `sim/demand.py:436 write_demand()`. Les types de véhicules :

▸ **`src/traffic_twin/sim/demand.py` : lignes 116–119** — *types SUMO : voiture, bus, ambulance*

```
116  VTYPES = """    <vType id="car" vClass="passenger" speedDev="0.1"/>
117      <vType id="bus" vClass="bus" length="12" maxSpeed="13.89" color="0,0.4,1"/>
118      <vType id="ambulance" vClass="emergency" guiShape="emergency" speedFactor="1.3"
119             color="1,0,0"/>"""
```

Voitures (calibrées sur les capteurs)

Un flux SUMO par branche × destination × tranche de 15 min (`sim/demand.py:247 slot_car_flows()`). Le débit vient du fichier de calibration `configs/calibration/pitie_302.yaml` (moyennes 07:30–09:30 : N ≈ 626, S ≈ 370, W ≈ 206 veh/h). On retire les bus déjà comptés par les capteurs, puis on multiplie par la part de mouvement tournant (supposée : N→S 0,63 / N→W 0,37 ; S→N 0,9 / S→W 0,1 ; W→N 0,85 / W→S 0,15). Les départs sont aléatoires (`probability`), la graine SUMO change donc les instants d'arrivée mais pas la demande moyenne.

Bus (`configs/pitie/demand.yaml:19-24`, horaires supposés)

Ligne	Itinéraire	Intervalle	Arrêt / temps d'arrêt
91_E / 91_W	wb_in→nb_out / nb_in→wb_out	360 s	stop_SM_E / stop_SM_W, 20 s
24_E / 24_W	wb_in→nb_out / nb_in→wb_out	600 s	stop_SM_E, 20 s / aucun
57_N / 57_S	s_in→nb_out / nb_in→s_out	480 s	stop_57_N / stop_57_S, 20 s

Code : `sim/demand.py:304 bus_line_flows()`.

Piétons

4 passages × 2 sens = 8 flux de 100 piétons/h chacun (`configs/pitie/demand.yaml:26`), générés par `sim/demand.py:329 crossing_walks()`.

Ambulance

Une ambulance par run principal à **08:10** (2400 s après le début), de la branche nord vers l'entrée sud de l'hôpital ; 7 ambulances dans la famille « probe ». Détails complets au chapitre 7.

6. Les contrôleurs de feux

Le contrat commun

Tout contrôleur hérite de `Controller` (`control/base.py:96`). À chaque pas il reçoit une `Observation` et renvoie **un seul entier : l'indice de l'étape verte souhaitée**. Il ne gère jamais les jaunes ni les rouges.

▸ **src/traffic_twin/control/base.py : lignes 44–52** — *ce que le contrôleur reçoit*

```
 44
 45
 46  @dataclass
 47  class Observation:
 48      t: float  # simulation time since scenario start (s)
 49      stage: int  # current green stage index; during a transition, the stage being LEFT
 50      time_in_stage: float  # seconds since the current green started (frozen during clearance)
 51      in_transition: bool  # True while clearance phases run
 52      queues: dict[str, int] = field(default_factory=dict)  # halting vehicles per approach
```

▸ **src/traffic_twin/control/base.py : lignes 113–115** — *ce que le contrôleur doit implémenter*

```
113  @abstractmethod
114  def decide(self, obs: Observation) -> int:
115      """Return the index of the desired green stage."""
```

Les `extras` contiennent : lacunes des boucles (`gap_s`), attente et marge des piétons (`ped_wait`, `ped_slack`), appels d'urgence (`emergency`), cible de préemption (`preempt`), appels bus (`bus_calls`), sorties bloquées (`exit_blocked`) — `control/base.py:15-41`.

S0 – Feux fixes

▸ **src/traffic_twin/control/fixed.py : lignes 29–32**

```
 29  def decide(self, obs: Observation) -> int:
 30      if obs.in_transition or obs.time_in_stage < self.green_s[obs.stage]:
 31          return obs.stage
 32      return (obs.stage + 1) % len(self.green_s)
```

Il garde l'étape tant que son temps de vert fixe n'est pas écoulé, puis passe à la suivante.

S1 – Actuated (détection de lacunes)

▸ **src/traffic_twin/control/actuated.py : lignes 72–82**

```
 72  def decide(self, obs: Observation) -> int:
 73      s = obs.stage
 74      if obs.in_transition or obs.time_in_stage < self.min_green_s[s] - _EPS:
 75          return s
 76      nxt = (s + 1) % len(self.min_green_s)
 77      if obs.time_in_stage >= self.max_green_s[s] - _EPS:
 78          return nxt  # max-out
 79      gaps = obs.extras.get(EXTRA_GAP_S)
 80      if gaps is None:
 81          raise ConfigError("actuated needs detectors.control: the observation has no gap_s")
 82      return s if gaps[s] < self.max_gap_s else nxt  # extend, else gap-out
```

Après un vert minimum de 6 s, il prolonge le vert tant que des véhicules passent sur la boucle amont (lacune < `max_gap_s` = 3 s) et coupe dès qu'un trou apparaît (**gap-out**) ou que le vert maximum est atteint (**max-out** : HOP 38, STM 22 s…). Paramètres : `configs/pitie/control.yaml:17-36` ; boucles à 16,66 m de la ligne d'arrêt. La lacune est lue dans SUMO par `sim/session.py:323-345` (`getTimeSinceDetection`).

S2 – Contrôleur par règles (solution principale)

Ordre de priorité des règles dans `decide()` :

- 1. tenir le vert minimum et ne jamais interrompre un dégagement ;
- 2. logique de lacunes **avec dégagement de la zone de conflit** (box) : on ne prolonge pas un vert si la sortie est bloquée ;
- 3. coupure anticipée pour un bus (implémentée, **désactivée** à Pitié) ;

- 4. **prolongation de vert pour un bus** qui arrive (jusqu'à +10 s) ;
- 5. **anticipation piétons** : quitter le vert avant que le bouclier piéton ne doive le forcer ;
- 6. sinon garder l'étape ou passer à la suivante.
- La **préemption ambulance** n'est pas dans le contrôleur : elle est appliquée au-dessus, dans le SafetyGuard (chapitre 7), donc elle a priorité sur toute décision du contrôleur.

▸ **src/traffic_twin/control/rules.py : lignes 280–330** *— la décision complète de S2*

```python
def decide(self, obs: Observation) -> int:
    self._track_green(obs)
    s, tis = obs.stage, obs.time_in_stage
    calls = self._tsp_calls(obs) if self.tsp.enabled else ()
    if obs.in_transition or tis < self.min_green_s[s] - _EPS:
        return s
    nxt = (s + 1) % len(self.min_green_s)
    mx = self.max_green_s[s]

    # 2. gap logic, with box clearing on the open-exit loops
    gaps = self._extra(obs, EXTRA_GAP_S, "detectors.control")
    if self.box:
        open_gaps = self._extra(obs, EXTRA_GAP_S_OPEN, "network.crossings (box clearing)")
        maxed = tis >= mx - _EPS
        stay = not maxed and open_gaps[s] < self.max_gap_s
        box_fired = not maxed and gaps[s] < self.max_gap_s <= open_gaps[s]
    else:
        stay = tis < mx - _EPS and gaps[s] < self.max_gap_s
        box_fired = False

    # 3. paid truncation (penalty cap) while a bus waits for another stage
    trunc = None
    if stay and self.tsp.truncation and calls:
        served_here = any(s in c.stages for c in calls)
        waiting = [c for c in calls if c.stages and s not in c.stages]
        cap = self.tsp.truncation_max_green_s[s]
        if waiting and not served_here and tis >= min(mx, cap) - _EPS:
            stay, trunc = False, waiting[0]

    # 4. TSP extension of a bus-serving stage about to end
    ext = None
    if not stay and trunc is None and s in self.tsp.extend_stages and calls:
        limit = mx + self.tsp.extension_max_s
        if tis < limit - _EPS:
            allow = limit - tis
            ok = [(self.tsp.eta(c), c.vid, c) for c in calls if s in c.stages]
            ok = [x for x in ok if x[0] <= allow + _EPS]
            if ok:
                stay, ext = True, min(ok)

    # 5. pedestrian anticipation: leave before the shield has to force it
    protect = False
    if stay and self.anticipate:
        slack = self._extra(obs, EXTRA_PED_SLACK, "network.crossings (pedestrian shield)")
        low = min(slack, default=math.inf)
        if low < self.dt - _EPS:
            stay, protect = False, True

    out = s if stay else nxt
    self._log(obs, out, gaps, box_fired, trunc, ext, protect)
    return out
```

Paramètres : `configs/pitie/rules.yaml:27-52` (verts max relâchés HOP 50 / STM 30, lignes prioritaires 91, 24, 57, `extension_max_s: 10`, préemption `max_hold_s: 120`).

S3 – Apprentissage par renforcement

▸ **src/traffic_twin/control/rl.py : lignes 280–296** — *décision en évaluation : réseau de neurones DQN*

```
280  def decide(self, obs: Observation) -> int:
281      """Clock every step; at decision points: sample -> observation -> predict."""
282      self._classify_last_step()
283      if not self.clock.is_decision_point(obs):
284          request = self.clock.request(obs, None, self.n_stages)
285      else:
286          if self.sampler is None:
287              raise RuntimeError("RLController.decide before bind(sim)")
288          x = observation(self.sampler.sample(), self.site, self.norm)
289          action, _state = self.policy.predict(x, deterministic=True)
290          action = int(np.asarray(action).item())
291          self.decisions += 1
292          self.advances += action
293          request = self.clock.request(obs, action, self.n_stages)
294          self.split.decided(obs, action, self.sim.guard.max_green_s)
295      self._last = (obs, request)
296      return request
```

Toutes les 5 s (après le vert minimum, hors préemption), le réseau reçoit 62 features et choisit **0 = rester** ou **1 = passer à l'étape suivante**. Le modèle chargé est `models/rl/rl7-dqn-v1-s0-415724a4/model.zip`, avec vérification de son empreinte SHA-256 (`control/rl.py:186-250`). Détails au chapitre 15.

S4 et les variantes « nopreempt »

`preemption: {enabled: false}` dans `configs/pitie_rl_s3_nopreempt.yaml:4` (S4) et `configs/pitie_rules_nopreempt.yaml:4` (S2_noP) : `PreemptionManager.from_config` renvoie alors `None` (`control/preemption.py:122-123`) et l'ambulance est traitée comme une voiture (elle est toujours détectée, mais personne n'agit). Ces variantes servent à **mesurer l'apport réel de la préemption**.

7. L'ambulance : de sa création à son temps de parcours

Résumé pour le professeur. L'ambulance est créée dans le fichier de demande (`sim/demand.py:191`), détectée à l'approche du feu (`sim/emergency.py:95`), sa demande est gérée en file « premier arrivé, premier servi » (`control/preemption.py:158`), le **SafetyGuard** impose l'étape verte qui la laisse passer par le chemin le plus court et sûr (`control/safety.py:322-325` puis `:367`), puis tout revient à la normale quand elle a franchi la ligne d'arrêt. Son temps est mesuré dans `sim/kpi.py:284` et testé statistiquement dans `evaluation/evaluate.py:1051` (critère A-05 : **PASS**, −36,5 s).

7.1 La chaîne d'appels complète (un pas de simulation)

#	Appel	Emplacement	Rôle
1	`run_scenario.run`	`sim/run_scenario.py:228`	boucle principale
2	`SimSession.observe`	`sim/session.py:261`	construit l'observation
3	`_junction_extras`	`sim/session.py:347-352`	interroge détecteur + préemption
4	`EmergencyDetector.calls`	`sim/emergency.py:95`	liste les ambulances en approche
5	`PreemptionManager.update`	`control/preemption.py:158`	choisit la demande active (FCFS)
6	`controller.decide(obs)`	ex. `control/rules.py:280`	le contrôleur propose une étape
7	`SimSession.step`	`sim/session.py:277`	transmet au SafetyGuard
8	`SafetyGuard.apply`	`control/safety.py:310`	règle R3 : préemption prioritaire
9	`_preempt` → `_preempt_hop`	`control/safety.py:367`, `:391`	prochaine étape vers la cible
10	`next_hop` (Dijkstra)	`control/preemption.py:48`	chemin le plus court et sûr
11	`PedestrianShield.stay_excess/jump_excess`	`control/pedestrian.py:275`, `:283`	ne jamais dépasser 90 s d'attente piéton
12	`SignalPlan.request` → `setPhase`	`sim/session.py:292-295`	appliqué dans SUMO

Chemins relatifs à `src/traffic_twin/`.

7.2 Création de l'ambulance

Le type `ambulance` est un véhicule SUMO de classe `emergency`, rouge, 30 % plus rapide que la limite (`speedFactor=1.3`) — voir l'extrait du chapitre 5. Son trajet et son heure sont dans la configuration :

▸ **configs/pitie/demand.yaml : lignes 38–41** — *1 ambulance à 08:10, nord → entrée sud de l'hôpital*

```
38  emergency:
39    enabled: true
40    depart_s: 2400 # 08:10
41    route: [n_in, s_out] # Austerlitz -> hospital main gate (south); ASSUMED
```

▸ **src/traffic_twin/sim/demand.py : lignes 191–198** — *écrit le <vehicle type="ambulance"> dans le .rou.xml*

```
191  def ambulance_elements(cfg: dict, begin: int) -> list[Element]:
192      """One `<vehicle type="ambulance">` per emergency trip, as sortable elements."""
193      out: list[Element] = []
194      for depart, vid, route in emergency_trips(cfg):
195          xml = _elem("vehicle", _route(route), id=vid, type="ambulance", depart=begin + depart,
196                      departLane="best")  # fmt: skip
197          out.append((begin + depart, KIND_ORDER["vehicle"], vid, xml))
198      return out
```

`emergency_trips()` (`sim/demand.py:149-188`) lit la config et renvoie `ambulance_0` (cas simple) ou `ambulance_0..6`. La famille **ambulance_probe** (`configs/pitie/emergency_probe.yaml:9-16`) injecte **7 ambulances** à 300, 1200, 2400, 3300, 4200, 5100 et 6000 s, alternant les trajets `n_in→s_out` et `w_in→s_out` ; elle est utilisée par `configs/pitie_amb_fixed.yaml`, `pitie_amb_rules.yaml`, `pitie_amb_rl_s3.yaml`.

7.3 Détection en amont

▸ **src/traffic_twin/sim/emergency.py : lignes 95–104** — *chaque pas : quelles ambulances approchent du feu ?*

```
 95  def calls(self, t: float) -> tuple[EmergencyCall, ...]:
 96      """Emergency calls at time t (since scenario start), in (first_seen_s, vid) order."""
 97      vehicle = self.traci.vehicle
 98      out: list[EmergencyCall] = []
 99      for vid in vehicle.getIDList():
100          if not self._emergency(vid):
101              continue
102          ahead = next((x for x in vehicle.getNextTLS(vid) if x[0] == self.tls_id), None)
103          if ahead is None:  # past the stop line (or never through this light)
104              continue
```

Un véhicule de classe `emergency` dont le prochain feu (`getNextTLS`) est notre carrefour génère un **appel** (identifiant, distance, lien emprunté, première détection). Les ambulances encore en attente d'insertion sont aussi signalées (`getPendingVehicles`, distance infinie) : cela modélise une détection amont de type V2I (`configs/pitie/rules.yaml:50`). Dès que l'ambulance a franchi la ligne d'arrêt, l'appel disparaît (ligne 103).

▸ **src/traffic_twin/sim/emergency.py : lignes 121–126** — *quelle étape verte sert le lien de l'ambulance ?*

```
121  def target_stage(self, link: int) -> int | None:
122      """Stage whose green phase shows 'G' on `link`, else 'g'; None if neither."""
123      if link not in self._stage:
124          full, permissive = link_stages(self.stages, self.phase_states, link)
125          self._stage[link] = (full or permissive or (None,))[0]
126      return self._stage[link]
```

7.4 Gestion de la demande de préemption

▸ **src/traffic_twin/control/preemption.py : lignes 158–186** — *FCFS, fin « servie » ou « timeout »*

```
158  def update(self, t: float, calls: Sequence[EmergencyCall]) -> int | None:
159      """Target stage of the active request at time t, or None.
160
161      FCFS on (first_seen_s, vid), dropping timed-out vids (t - last_progress_s >
162      max_hold_s); a vid seen before and absent now is 'served'."""
163      present: dict[str, EmergencyCall] = {}
164      for c in calls:
165          if c.vid in self._ended or c.vid in present:
166              continue
167          present[c.vid] = c
168      for vid in [v for v in self._open if v not in present]:
169          self._end(t, "served", self._open.pop(vid))
170      active: list[EmergencyCall] = []
171      for vid, c in present.items():
172          if vid not in self._open:
173              self.requests += 1
174              self._log(t, "request", c)
175          self._open[vid] = c
176          if t - self._last_progress(t, c) > self.max_hold_s:
177              self._end(t, "timeout", self._open.pop(vid))
178          else:
179              active.append(c)
180      if not active:
181          self.target = None
182          return None
183      best = min(active, key=lambda c: (c.first_seen_s, c.vid))
184      self.target = best.stage
185      self.active_s += self.dt
186      return best.stage
```

- Une ambulance qui disparaît des appels est marquée **servie** (`served`).
- Si elle ne progresse plus d'au moins 1 m pendant `max_hold_s` = 120 s, la demande expire (`timeout`) pour ne pas bloquer le carrefour indéfiniment.
- S'il y a plusieurs ambulances : premier détecté, premier servi.
- Tous les événements (`request`, `served`, `timeout`) sont journalisés dans `events.parquet`.

7.5 Comment la phase verte est imposée

▸ **src/traffic_twin/control/safety.py : lignes 310–328** — *SafetyGuard.apply : la règle R3 passe avant le contrôleur*

```
310  def apply(self, proposed: int, state: SignalState) -> int:
311      """The stage to request instead of `proposed`, given the plan state (D20)."""
312      if not 0 <= proposed < self.n_stages:  # R0
313          raise ValueError(f"unknown stage {proposed}")
314      target = state.preempt_target
315      if target is not None and not 0 <= target < self.n_stages:
316          raise ValueError(f"unknown preemption target {target}")
317      self._track(state)
318      self._state = state
319      s, tis = state.stage, state.time_in_stage
320      if state.in_transition:  # R1: SignalPlan ignores requests during clearance
321          return s
322      if target is not None:
323          if tis < self.min_green_s - _EPS:  # R2, hard; not an intervention
324              return s
325          return self._preempt(target, state)  # R3 (R4 suspended, R5 inside)
326      if proposed != s and tis < self.min_green_s - _EPS:  # R2
327          return self._intervene("min_green", s, f"proposed={proposed} green={tis:g}s")
328      return self._protect(self._part3(proposed, state), state)  # R4, then R5
```

Si une cible de préemption existe, la proposition du contrôleur est **ignorée** : seul le vert minimum (R2) et le dégagement en cours (R1) sont respectés. Ensuite :

▸ **src/traffic_twin/control/safety.py : lignes 367–395** — *_preempt : prochain saut vers l'étape cible*

```
367  def _preempt(self, target: int, state: SignalState) -> int:
368      """R3: next hop to the target, checked by the shield; the successor otherwise."""
369      s, succ = state.stage, self._succ(state.stage)
370      h = self._preempt_hop(s, target)
371      shield = self.shield
372      if h == s:
373          excess = () if shield is None else shield.stay_excess(state)
374      elif h == succ:
375          excess = ()
376      else:
377          excess = () if shield is None else shield.jump_excess(state, h)
378      if all(x <= _EPS for x in excess):
379          if h == s:
380              self.preemption["hold_s"] += self.dt
381          elif h != succ:
382              self.preemption["jumps"] += 1
383          return h
384      name, over = shield.worst(excess)
385      if self._once("ped_yield"):
386          self.preemption["ped_yield"] += 1
387          detail = f"target={target} hop={h} crossing={name} over={over:g}s"
388          self._log("ped_yield", succ, detail)
389      return succ
390
391  def _preempt_hop(self, s: int, target: int) -> int:
392      """`control.preemption.next_hop` over the link-safe graph, also under strict
393      order (D20): cost = clearance + min green per hop."""
394      clearance = self.clearance_s or [0.0] * self.n_stages
395      return next_hop(s, target, self.allowed, clearance, self.min_green_s)
```

`next_hop()` (`control/preemption.py:48-85`) fait un **Dijkstra** sur le graphe des transitions sûres (coût = dégagement + vert minimum de chaque étape traversée) : on peut donc **sauter** des étapes si c'est sûr. Le **bouclier piéton** vérifie chaque saut : si un piéton dépasserait 90 s d'attente, la préemption cède (`ped_yield`). Pour le RL, l'horloge de décision est suspendue pendant la préemption (`rl/clock.py:73`).

7.6 Retour à la normale

Quand l'ambulance passe la ligne d'arrêt, `getNextTLS` ne renvoie plus le feu → la demande est « servie » → `update()` renvoie `None` → le SafetyGuard redonne la main au contrôleur (`control/safety.py:326-328`). Une ambulance servie n'est jamais redemandée.

7.7 Mesure du temps de l'ambulance (KPI)

▸ **src/traffic_twin/sim/kpi.py : lignes 284–294** — *travel_time_total et stop_duration_mean*

```
284  amb = trips[trips["type"] == "ambulance"]
285  if not amb.empty:
286      add(
287          "ambulance", "travel_time_total", (amb["duration_s"] + amb["depart_delay_s"]).mean()
288      )
289      n_stops = amb["stops"].sum()
290      add(
291          "ambulance",
292          "stop_duration_mean",
293          amb["waiting_s"].sum() / n_stops if n_stops else 0.0,
294      )
```

`travel_time_total` = durée du trajet + retard au départ = **arrivée − départ prévu**. Les données viennent de `tripinfo.xml` (`sim/kpi.py:46-82`). Les compteurs de préemption (demandes, servies, timeouts, durée de maintien, sauts) sont écrits dans `run.json` (`sim/run_scenario.py:108-111`).

7.8 Preuve statistique : critère A-05

Test **pré-enregistré** (`configs/evaluation.yaml:25-33`) : sur les 10 graines, S2 réduit-il le temps de l'ambulance par rapport à S0 ? Test de **Wilcoxon exact unilatéral** sur les différences appariées + estimateur de **Hodges-Lehmann** (`evaluation/evaluate.py:1051 a05_test`). Le test est déclaré invalide si une graine manque ou si une ambulance n'arrive pas (`evaluate.py:1027`).

▸ **src/traffic_twin/evaluation/evaluate.py : lignes 1094–1094** — *condition de réussite*

```
1094  "pass": bool(not problems and w is not None and p < alpha),
```

Comparaison	Moyenne avant → après	Effet (HL) [IC 95 %]	p	Verdict
S2 vs S0, famille principale (1 ambulance/run)	152,2 s → 110,5 s	−36,5 s [−70,0 ; −14,0]	0,0020	**PASS**
S2 vs S0, famille probe (7 ambulances/run)	179,8 s → 87,0 s	−83,1 s	0,0010	PASS

Préemptions servies : 10/10 (principale) et 70/70 (probe), 0 timeout. Sans préemption (S2_noP) l'ambulance met 134,4 s contre 110,5 s avec : c'est bien la préemption qui fait le gain (source : `docs/evaluation.md`).

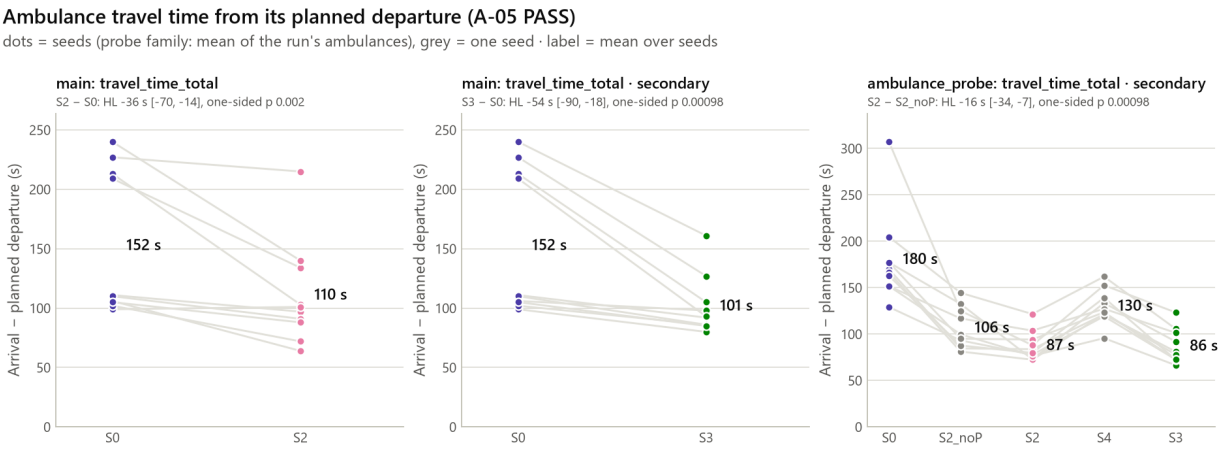


Figure 2 – A-05 : temps de l'ambulance par graine (`docs/figures/pitie/eval/a05_ambulance.png`)

7.9 Montrer l'ambulance en direct (3 façons)

a) Dans SUMO avec l'interface graphique

```
.venv\Scripts\python.exe tasks.py sim-gui --scenario configs/pitie_amb_rules.yaml --seeds 101
```

La famille *probe* fait passer 7 ambulances (la première à 07:35). L'ambulance est le véhicule **rouge** à forme d'urgence (`guiShape="emergency"`), identifiants `ambulance_0` à `ambulance_6` (menu *Locate → Vehicles* de sumo-gui). Avec `configs/pitie_rules.yaml`, il n'y en a qu'une, à 08:10. Comparer avec `configs/pitie_amb_rules_nopreempt.yaml` pour voir la différence sans préemption.

b) Dans l'application web (démo)

Après `python tasks.py bootstrap` puis `python tasks.py demo` (chapitre 18) :

- Vue ambulance : `http://127.0.0.1:8000/mock?view=ambulance&scenario=pitie_rules&seed=4000&t=2440&kiosk=1` → le carrefour affiche **ONDE VERTE** (préemption en cours, feu vert).
- Replay 2D/3D : `http://127.0.0.1:8000/mock?view=replay&scenario=pitie_rl_s3&seed=4000&t=2440` → ambulance blanche à bandes rouges et gyrophare bleu/rouge.

Où dans src/traffic_twin/app/mock.html	Quoi
:676-698	CSS du HUD urgence et du clignotement sirène
:1804-1817 vehicleOf()	reconnaissance du type ambulance et de sa taille
:1895-1896	isEmergency : une ambulance est-elle sur la carte ?
:2076-2098	pavillon « URGENCES & SAMU 75 » qui clignote pendant le passage
:2533-2550	dessin 2D de l'ambulance (bandes rouges, gyrophare rotatif)
:3098-3100, :3126-3145	modèle 3D (three.js) et rotation du gyrophare
:3616-3654 ambulance()	vue ambulance : DÉGAGÉ / ONDE VERTE / BLOQUÉ

Le statut affiché est calculé par l'API (`GET /ambulance/next-intersections`, `api/main.py:1147`) :

▸ **src/traffic_twin/api/main.py : lignes 1378–1392** *— calcul du statut ONDE VERTE*

```
1378  status, reason = "unknown", "pas de données de file"
1379  if tick is not None and passed:
1380      status, reason = "clear", "carrefour franchi"
1381  elif tick is not None:
1382      reqs = open_requests(store.events(run.run_id), tq)
1383      requested = (vid in reqs) if vid is not None else bool(reqs)
1384      req_link = reqs.get(vid) if vid is not None else None
1385      links = (req_link,) if req_link is not None else ()
1386      if not links and mv and mv[1]:
1387          links = site.movement_links.get(mv, ())
1388      green = links_green(store.signal(run.run_id), links, tq)
1389      if requested and green:
1390          status, reason = "green_wave", "préemption en cours, feu vert"
1391      else:
1392          notes = []
```

c) Par les tests automatiques (preuve exécutable)

`.venv\Scripts\python.exe -m pytest tests/test_pitie_e2e.py -k ambulance -v` lance une vraie simulation SUMO et vérifie que S2 préempte (1 demande, 1 servie, 0 timeout — `tests/test_pitie_e2e.py:219`) et que l'ambulance n'est pas plus lente qu'en S0 (`:245`).

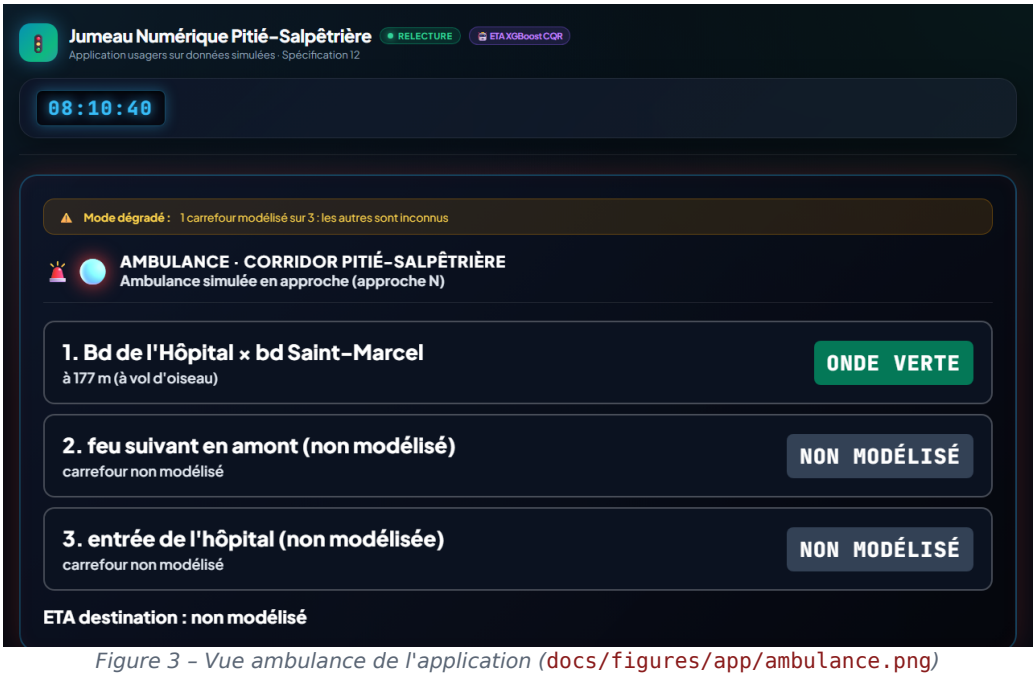


Figure 3 – Vue ambulance de l'application (`docs/figures/app/ambulance.png`)

7.10 Tests qui couvrent l'ambulance

Fichier	Tests (ligne)
`tests/test_emergency.py`	détection et étape cible (:58), fin après la ligne d'arrêt (:69), véhicules en attente (:81), ordre FCFS (:97), détection sur le réseau Pitié (:148)
`tests/test_preemption.py`	next_hop sur le programme Pitié (:58), chemin le moins cher (:64), FCFS (:113), servie (:124), timeout à max_hold (:142), timeout relancé par la progression (:155)
`tests/test_safety.py`	la préemption ne coupe jamais vert min/dégagement (:414), suspend les règles Partie 3 (:437), cède au bouclier piéton (:449, :464)
`tests/test_demand.py`	ambulances triées et numérotées (:496, :516), demande probe chargée dans SUMO (:597)
`tests/test_kpi.py`	KPI ambulance (:332), ambulance sans arrêt (:338)
`tests/test_evaluate.py`	A-05 : p = 1/1024 (:182), graine manquante → invalide (:681), ambulance non arrivée → invalide (:700)

8. La priorité bus (TSP)

Détection : `BusDetector` (`sim/transit.py:30`, `calls()` ligne 70) repère les bus sur les branches d'entrée, leur distance au feu et le temps d'arrêt restant à la station.

Décision (dans S2, règle 4) : si le vert d'une étape qui sert le bus allait se terminer et que le bus peut atteindre la ligne d'arrêt dans la prolongation autorisée (+10 s max), le vert est prolongé. Le temps d'arrivée estimé :

▸ **src/traffic_twin/control/rules.py : lignes 73–79** — *ETA du bus jusqu'à la ligne d'arrêt*

```
73  def eta(self, call: BusCall) -> float:
74      """Seconds until the bus can reach the stop line: distance at the approach speed,
75      plus the remaining dwell, plus the start penalty when halted or dwelling."""
76      eta = call.dist_m / self.approach_speed_mps + call.dwell_left_s
77      if call.speed_mps < HALTED_MPS or call.dwell_left_s > 0:
78          eta += self.start_penalty_s
79      return eta
```

Configuration : `configs/pitie/rules.yaml:34-46` (lignes 91, 24, 57 ; étapes S_LEFT, BUS_91, BUS_57 ; coupure anticipée désactivée). La TSP n'utilise pas les horaires : la régularité est mesurée comme KPI (`headway_std`, `sim/kpi.py:296-304`).

Résultat (10 graines) : retard bus moyen 61,0 s (S0) → 56,1 s (S2, −8 %) → 47,6 s (S3, −22 %). Tests : `tests/test_transit.py`, `tests/test_rules.py:143-197`.

9. Les piétons et le bouclier piéton

Garantie : aucun piéton n'attend plus de **90 s** au bord du trottoir (`configs/pitie/control.yaml:57-59`, marge 2 s → seuil interne θ = 88 s). Le `PedestrianShield` (`control/pedestrian.py:108`) vérifie l'invariant *attente actuelle + temps jusqu'au prochain vert piéton* ≤ *θ* avant chaque décision :

▸ **src/traffic_twin/control/pedestrian.py : lignes 275–288**

```
275  def stay_excess(self, state: SignalState) -> tuple[float, ...]:
276      """Per crossing: w_c + T_c^fb + dt - theta (a stay is allowed iff all <= 0)."""
277      s, tis = state.stage, state.time_in_stage
278      return tuple(
279          w + self.time_to_green(c, s, tis) + self.dt - self.threshold_s
280          for c, w in enumerate(self.waits(state))
281      )
282
283  def jump_excess(self, state: SignalState, nxt: int) -> tuple[float, ...]:
284      """Per crossing: w_c + T_via(c, s, nxt) - theta (a jump is allowed iff all <= 0)."""
285      s = state.stage
286      return tuple(
287          w + self.time_via(c, s, nxt) - self.threshold_s for c, w in enumerate(self.waits(state))
288      )
```

- Si garder le vert ferait dépasser θ, le SafetyGuard force l'étape suivante (`control/safety.py:349-365`, intervention `ped_wait`).
- S2 anticipe en quittant le vert un peu plus tôt (`control/rules.py:321-326`).
- L'attente est mesurée au trottoir sans être remise à zéro par les petits mouvements (`sim/session.py:375-421`).
- Au démarrage, une configuration où un passage piéton pourrait dépasser θ est **refusée** (`control/pedestrian.py:173-214`).

Résultat : attente maximale observée **88 s** sur 110 runs (borne 90 s). Tests : `tests/test_pedestrian.py`, `tests/test_guard_fuzz.py:120` (contrôleurs adverses aléatoires qui ne cassent jamais la borne).

10. Le SafetyGuard : la sécurité avant tout

Le SafetyGuard (`control/safety.py:156`) est placé **entre n'importe quel contrôleur et SUMO** (`sim/session.py:290`). Même un RL qui ferait n'importe quoi ne peut pas produire une séquence dangereuse. Règles appliquées dans l'ordre :

Règle	Effet	Code
R0	étape inconnue → erreur	`safety.py:312-316`
R1	pendant un jaune/rouge de dégagement : on garde l'étape courante	`safety.py:320-321`
R2	vert minimum 6 s	`safety.py:326-327`
R3	préemption ambulance (prioritaire)	`safety.py:322-325`, `_preempt :367`
R4	anti-famine (rouge max 150/180 s), ordre strict du cycle, vert max 60 s	`_part3 safety.py:331-347`
R5	bouclier piéton 90 s	`_protect safety.py:349-365`

En cas de violation, le guard **remplace** la demande par une étape sûre et journalise une intervention (il n'arrête pas la simulation). Les transitions autorisées sont uniquement celles du graphe « link-safe » (`safety.py:61-89`) : jamais de vert → rouge sans jaune, jamais jaune → vert.

Audit indépendant après chaque run (A-04)

`sim/audit.py:65 audit_phases()` relit les phases **réellement affichées** et compte : vert→rouge sans jaune, dégagement incomplet, intervert trop court, verts antagonistes, vert < minimum, transition illégale, phase inconnue. Le critère A-04 (`evaluation/evaluate.py:903`) exige 0 collision, 0 séquence dangereuse, attente piéton ≤ 90 s et 0 intervention pour S0/S1.

> **A-04 : PASS sur 110 runs** — 0 collision, 0 téléportation, 0 séquence dangereuse, attente piéton maximale 88 s.

11. Blocage du carrefour et incidents

JunctionMonitor (`sim/junction.py:83`)

Observateur passif appelé à chaque pas (`sim/session.py:307-308`). Il mesure : véhicules arrêtés dans la zone de conflit pendant leur rouge (`box_halt_red_s`), sorties bloquées (véhicule arrêté à moins de 15 m du début de la voie de sortie), piétons entrant au rouge et piétons exposés à un vert véhicule antagoniste (`crossing_complete_rate`). Les sorties bloquées alimentent la règle « box » de S2 (`control/rules.py:291-295`) : on ne prolonge pas un vert qui enverrait des voitures dans une sortie saturée.

Incidents (`sim/incidents.py`)

Un incident = réduction de vitesse d'une voie à 2 m/s pendant une durée tirée au hasard (`IncidentManager.update`, ligne 131 ; sites dans `configs/pitie/incident_sites.yaml`). Ils sont utilisés **uniquement** pour générer les mondes du modèle ETA et les épisodes d'entraînement RL (robustesse), **pas** dans les scénarios d'évaluation S0–S4.

12. Les indicateurs (KPI) par type d'usager

Usager	KPI principaux	Code
Voiture	temps perdu moyen, attente moyenne et p95, arrêts, débit (veh/h), retard total par branche	`sim/kpi.py:155-164`, `:260-274`
Bus	retard moyen, régularité (écart-type des intervalles), temps d'arrêt	`sim/kpi.py:165-178`, `:275-304`
Ambulance	temps total (arrivée − départ prévu), nombre et durée des arrêts	`sim/kpi.py:284-294`
Piéton	attente au trottoir moyenne / p95 / max, par passage, taux de traversée sûre	`sim/kpi.py:180-191`, `:306-325`
Tous	**retard par personne** = Σ(occupation × retard) / Σ occupation	`evaluation/evaluate.py:746`

Le retard par personne pondère chaque véhicule par son nombre d'occupants (`configs/evaluation.yaml:48` : voiture 1,3 ; bus 25 ; piéton 1 ; ambulance 1) : un bus retardé pèse autant que 25 personnes. C'est le KPI principal de la comparaison RL (A-06).

13. Données, choix du carrefour, calibration et validation

Source et nettoyage

API Open Data Paris, jeu « comptages-routiers-permanents » (`configs/data.yaml:1-8`, 12 mois). Téléchargement filtré par distance au carrefour (`data/download.py:22-52`). Nettoyage (`data/clean.py`) : horodatage = **fin** de l'heure mesurée (corrigé par `data/calendar.py:37`), débits hors [0, 4000] veh/h ou capteur « Barré/Invalide » rejetés, doublons supprimés, couverture < 80 % signalée.

Choix du carrefour (score 79,3 %)

▸ **src/traffic_twin/data/intersection_score.py : lignes 28–35**

```
28  weighted = sum(weights[c] * scores[c] for c in weights)
29  rows.append(
30      {
31          "candidate": key,
32          "label": cand.get("label", key),
33          "score_pct": round(100 * weighted / (MAX_SCORE * total_weight), 1),
34          "eliminated": any(scores[c] == 0 for c in eliminatory),
35      }
```

8 critères pondérés (`configs/intersections.yaml:4-13`) : capteur (5, éliminatoire), feux 3–4 branches (4), ligne de bus (4), passage piéton (4), urgences < 1 km (3), géométrie simple (3), université/gare (2), filmable (2). Pitié : 107 / (5 × 27) = **79,3 %** ; les deux autres candidats sont éliminés faute de capteur.

Calibration de la demande

- Jours types : mardi-mercredi-jeudi en période scolaire, hors fériés (`data/calendar.py:137-153`), filtre robuste médiane ± 3,5 MAD (`data/calendar.py:251-263`).
- Fenêtre 07:30–09:30 en tranches de 15 min (`configs/data.yaml:82`).
- Débit par branche : `data/calibrate.py:377 build_calibration()` (au moins 8 jours).
- Boucle de calibration simulation ↔ réalité (`validation/calibration_loop.py:62`) : ajuste un facteur par branche jusqu'à ±5 %. Résultat : **convergée sans itération** (erreurs −1,5 %, −2,8 %, −0,1 % ; `docs/calibration_log.yaml`).

▸ **src/traffic_twin/validation/calibration_loop.py : lignes 88–91** — *critère de convergence*

```
88  for a in arms:
89      demand = base_demand[a] * scale[a]
90      err = sim[a] / real[a] - 1
91      within[a] = abs(err) <= tolerance + 1e-12
```

Validation simulation vs réalité (spec 13.4)

▸ **src/traffic_twin/evaluation/stats.py : lignes 245–248** — *statistique GEH*

```
245  def geh(model: float, count: float) -> float:
246      """GEH on hourly flows (veh/h): sqrt(2 (M - C)^2 / (M + C)); 0 when both are 0."""
247      s = model + count
248      return 0.0 if s <= 0 else math.sqrt(2.0 * (model - count) ** 2 / s)
```

Contrôle	Règle	Résultat
Débits (A-02)	écart ≤ ±15 % par branche × heure ; GEH < 5 sur ≥ 85 % des comptages	**PASS** : 6/6 lignes dans ±15 % ; GEH < 5 sur 10/10
Temps de feux	cycle ±2 s, vert ±2 s, jaune/rouge ±0,5 s	**NON MESURÉ** (visite terrain)
Files d'attente	file observée dans [p10, p90] simulé	**NON MESURÉ** (visite terrain)

Codes : `validation/flows.py:112` (débits), `validation/signal.py:312` (feux), `validation/queues.py:74` (files), rapport `validation/report.py:113` → `docs/validation.md`. Limite reconnue : la demande étant construite à partir des mêmes capteurs, A-02 n'est pas une preuve indépendante de réalisme (`docs/validation.md:116`).

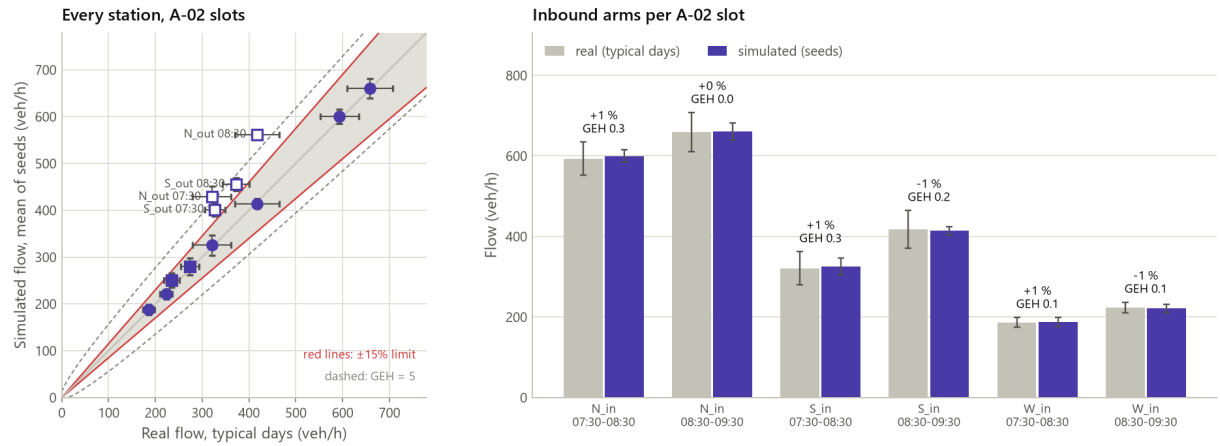


Figure 4 – Débits simulés vs réels (`docs/figures/pitie/validation_flows.png`)

14. Le modèle de temps de parcours (ETA)

Question à laquelle il répond (vue étudiant) : « si j'entre maintenant par N pour aller vers S, combien de temps vais-je mettre, avec quelle incertitude ? »

- **Données** : 300 « mondes » simulés d'entraînement + validation/calibration/test séparés par graine (`configs/eta.yaml:8`), chacun avec demande, type de jour, ambulances et incidents aléatoires (`ml/eta_dataset.py:178 sample_world()`). Cible : temps passé dans la zone (`ml/eta_dataset.py:432`).
- **Features** : files, attentes, état des feux, heure... sans regarder le futur (`ml/features.py:82-94`, jointure `merge_asof` arrière `:338-363`).
- **Modèle** : XGBoost à quantiles 10 / 50 / 90 % (`ml/train_eta.py:339 fit_quantile_model()`).
- **Intervalle conforme** : un décalage q̂ calibré pour que l'intervalle 80 % couvre vraiment 80 % (`ml/train_eta.py:420 fit_conformal()`).

▸ **src/traffic_twin/ml/train_eta.py : lignes 407–417** — *bornes servies à l'utilisateur*

```
407  def served_bounds(pred: np.ndarray, qhat: float) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
408      """(low, median, high) whole seconds: floor(q10 - qhat), round(q50), ceil(q90 + qhat),
409      clipped to 0 <= low <= median <= high (D48). Rows of `pred` are sorted first (quantile
410      crossing repair); `round` is half-to-even (numpy `rint`)."""
411      p = np.sort(np.asarray(pred, dtype=float).reshape(-1, 3), axis=1)
412      if not np.isfinite(p).all():
413          raise ValueError("served_bounds: non-finite quantile prediction")
414      median = np.maximum(np.rint(p[:, 1]), 0.0)
415      low = np.minimum(np.maximum(np.floor(p[:, 0] - qhat), 0.0), median)
416      high = np.maximum(np.ceil(p[:, 2] + qhat), median)
417      return low.astype(np.int64), median.astype(np.int64), high.astype(np.int64)
```

A-07 : PASS — couverture 80,6 % [80,2 ; 81,0] (cible 75–85 %) ; erreur absolue moyenne **10,2 s** contre 39,6 s pour la référence (moyenne par trajet/heure/jour) ; 483 100 trajets de test. Seul point faible : couverture 71,9 % en présence d'un incident (`docs/eta.md`).

Conseil de départ pour l'étudiant

▸ **src/traffic_twin/ml/advice.py : lignes 70–76**

```
70  if arrival_high <= deadline:
71      advice = Advice.COMFORTABLE
72  elif arrival_median <= deadline:
73      advice = Advice.SMALL_MARGIN
74  else:
75      advice = Advice.RISK
76  message = MESSAGES_FR[advice] + (APPROXIMATE_SUFFIX if approximate else "")
```

Messages : « Partez maintenant, marge confortable » / « Marge faible » / « Risque de retard, prévenez ». Appelé par `GET /eta` (`api/main.py:1152`).

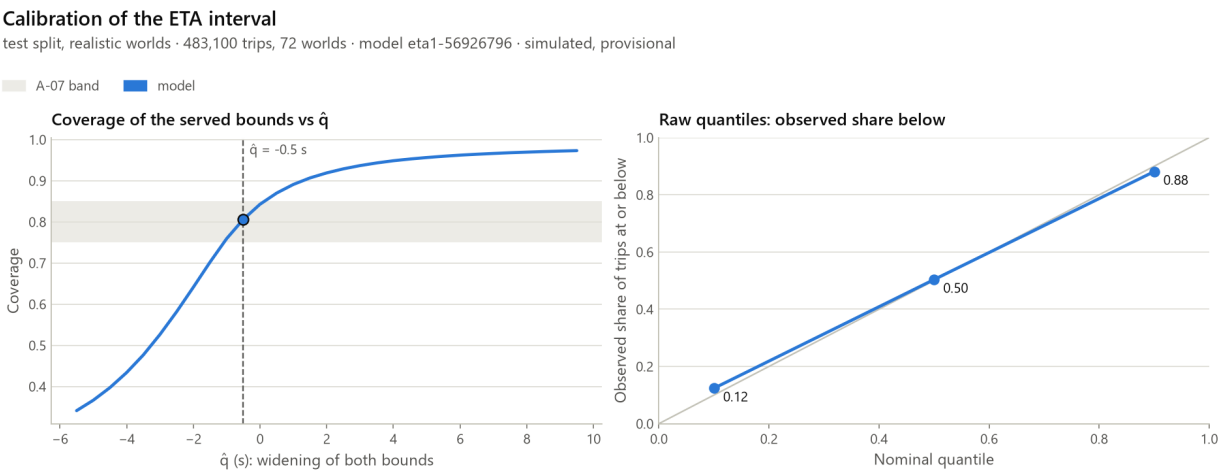


Figure 5 – Calibration de l'intervalle ETA (`docs/figures/eta/calibration_curve.png`)

15. L'apprentissage par renforcement (S3)

L'environnement Gymnasium (`rl/env.py:347 TrafficEnv`)

Élément	Choix	Code
Action	`Discrete(2)` : 0 = rester sur l'étape, 1 = passer à la suivante	`rl/env.py:390`, `rl/clock.py:89`
Pas de décision	toutes les 5 s après le vert minimum, jamais pendant une préemption	`rl/clock.py:69-75`, `configs/rl.yaml:5`
Observation	74 valeurs dans [0,1] (62 pour S3, sans le groupe piéton) : files/véhicules/attente par voie, étape courante, rouges, piétons, ambulance (présence, distance, branche), bus (présence, distance, retard), heure (sin/cos)	`rl/features.py:213-273`
Épisode d'entraînement	fenêtre de 30 min après 5 min de chauffe, demande ×U(0,6 ; 1,4), 0–2 ambulances, incident avec p = 0,3	`rl/env.py:102-266`, `configs/rl.yaml:16-22`
Sécurité	l'action passe par le même SafetyGuard que les autres contrôleurs	`sim/session.py:290`

La récompense

▸ **`src/traffic_twin/rl/reward.py` : lignes 107–113** — *coût = attente pondérée par type d'usager*

```
107  cost = dt * (
108      w["car"] * n["car"]
109      + w["bus"] * n["bus"]
110      + w["pedestrian"] * n_ped
111      + w["ambulance"] * n["ambulance"]
112      + (self.lam if over else 0.0)
113  )
```

Poids (`configs/rl.yaml:11-14`) : voiture 1,3 ; bus 25 ; piéton 1,2 ; ambulance 20 ; pénalité λ = 30 dès qu'un piéton attend plus de 80 s. Récompense = −0,001 × coût.

Entraînement et sélection

- Algorithme **DQN** (Stable-Baselines3, `MlpPolicy` 128×128), `rl/train.py:584` ; taux d'apprentissage décroissant 5e-4 → 5e-5, γ = 0,95, buffer 100 000.
- Campagne Partie 7 (`rl/campaign.py`, `configs/rl7.yaml`) : étape T (5 configurations × 3 graines × 400 k pas), étape F (5 graines × 1 M pas), étude de sensibilité aux poids (9 variantes). Budget total ≈ 24,2 M pas.
- Règles de sélection pré-enregistrées : `rl/study.py:523` (config) et `:1067` (modèle). Modèle retenu : `models/rl/rl7-dqn-v1-s0-415724a4` (pas 975 000).
- Commande : `python -m traffic_twin.rl.train --config configs/rl.yaml --algo dqn --seed 0`.

Le verdict « FALLBACK » (spec 10.6), à bien expliquer

Critère	Test	Résultat (graines 1–10)	OK ?
C1	retard par personne S3 vs S1	HL −9,5 s, p = 0,002	oui
C2	équité : aucun KPI > 5 % pire que S1	voiture −14,5 %, bus −23,9 %, **piéton +5,2 %**	**non**
C3	sécurité	0 violation	oui
C4	reproductibilité	5/5 modèles meilleurs que S1 sur données réservées	oui

Le RL est **meilleur globalement** (il bat S1, S2 et S0 sur le retard par personne), mais il dégrade un peu l'attente des piétons (+5,2 % vs S1, seuil 5 %). La règle pré-enregistrée impose donc de **garder S2 comme solution principale** et de présenter le RL comme une étude. Raison probable : la variante retenue (V1) ne voit pas les piétons ; seul le bouclier les protège. Code du verdict : `rl/study.py:1732 compute_verdict()`.

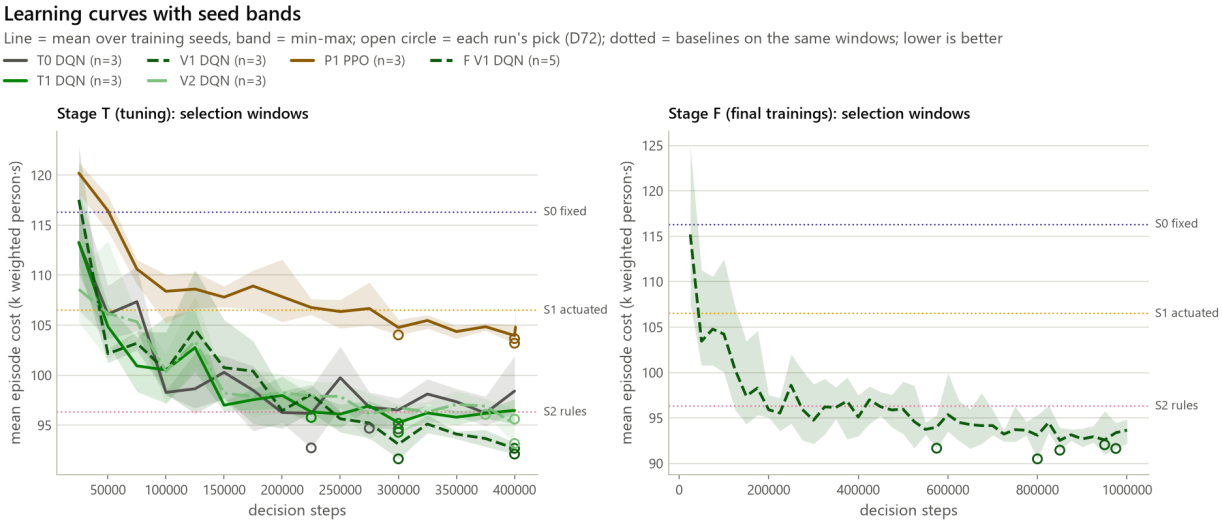


Figure 6 – Courbes d'apprentissage RL (`docs/figures/rl7/learning_bands.png`)

16. Évaluation finale S0 à S4

Protocole (pré-enregistré le 2026-10-12)

- 10 graines (1–10) par scénario, mêmes graines pour tous → comparaisons **appariées**.
- Famille principale (1 ambulance) + famille probe (7 ambulances) : 110 runs au total.
- Tests : **Wilcoxon signé exact** (`evaluation/stats.py:69`), **Hodges-Lehmann** + IC (`stats.py:215`), correction de **Holm** pour comparaisons multiples (`stats.py:233`).
- Entrée : `python tasks.py evaluate` → `evaluation/evaluate.py:2580` ; tableau final `evaluation/final.py:748` ; rapport `docs/evaluation.md`.

Tableau final (moyennes sur 10 graines)

KPI	S0 fixe	S1 actuated	S2 règles	S3 RL	S2_noP	S4
Voiture : temps perdu (s)	75,9	65,8	56,6	56,2	55,9	56,1
Voiture : attente p95 (s)	164,5	140,0	111,6	113,7	109,6	112,9
Voiture : débit (veh/h)	1178,8	1179,5	1185,7	1185,7	1184,7	1185,3
Bus : retard moyen (s)	61,0	62,6	56,1	47,6	57,2	47,4
Piéton : attente moy. (s)	18,9	19,2	19,4	20,2	19,1	20,1
Piéton : attente max (s)	85,6	87,4	88,0	88,0	88,0	88,0
Ambulance : temps total (s)	152,2	149,1	**110,5**	**101,3**	134,4	125,3
Retard par personne (s)	63,6	57,1	**48,9**	**46,0**	48,9	45,9

Lecture : S2 réduit le retard par personne de 23 % par rapport au feu fixe et le temps de l'ambulance de 27 % ; S3 fait légèrement mieux globalement, mais pénalise un peu les piétons. Désactiver la préemption (S2_noP, S4) rallonge nettement le trajet de l'ambulance.

A-06 : le RL comparé aux références

Comparaison	Réf. → S3 (s)	HL [IC 95 %]	p Holm	Issue
S3 vs S1 (primaire)	57,1 → 46,0	−9,5 [−17,5 ; −6,4]	0,0059	S3 meilleur
S3 vs S2	48,9 → 46,0	−2,5 [−5,0 ; −0,6]	0,0137	S3 meilleur
S3 vs S0	63,6 → 46,0	−14,3 [−29,4 ; −10,7]	0,0059	S3 meilleur

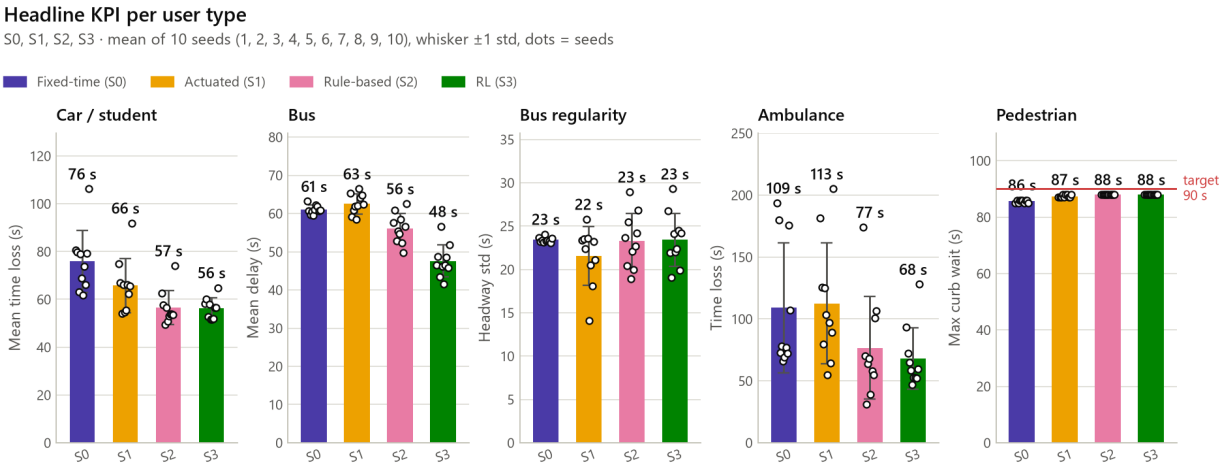


Figure 7 – KPI par usager et par contrôleur (`docs/figures/pitie/eval/kpi_by_user.png`)

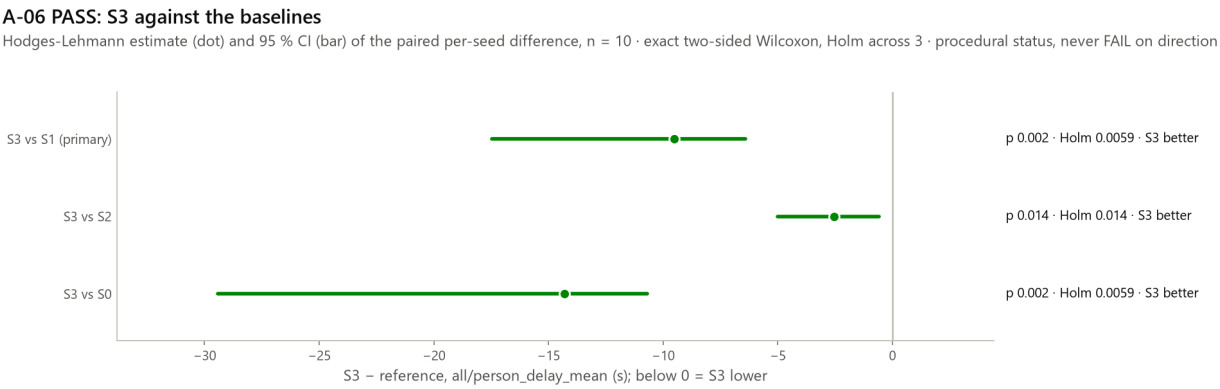


Figure 8 – A-06 : effets estimés (Hodges-Lehmann) (`docs/figures/pitie/eval/a06_forest.png`)

17. L'API et l'interface

API REST FastAPI (`src/traffic_twin/api/main.py`, `create_app()` ligne 919)

Méthode + chemin	Handler (ligne)	Usage
GET `/health`	:961	état du service
GET `/scenarios`	:977	scénarios et graines disponibles
GET `/kpi`	:1034	KPI d'un run
GET `/replay/{run_id}`	:1043	positions et feux à l'instant t (replay)
GET `/replay/{run_id}/events`	:1188	événements (préemption, priorité bus)
GET `/crossing/{crossing_id}`	:1094	vue piéton : attente avant le vert
GET `/bus/{bus_id}/status`	:1142	vue bus : retard, priorité
GET `/ambulance/next-intersections`	:1147	vue ambulance : DÉGAGÉ / ONDE VERTE / BLOQUÉ
GET `/eta`	:1152	vue étudiant : ETA + intervalle + conseil
GET `/evaluation`, `/compare`, `/series`, `/sensitivity`	:1163–1184	résultats d'évaluation
GET `/mock`, `/`	:1198, :1219	sert l'interface web

Une erreur interne ne renvoie jamais 500 mais 503 avec un message (`never_500`, ligne 933). Chaque réponse porte la mention « données simulées ».

Interface web `src/traffic_twin/app/mock.html` (servie par l'API)

Vue	Fonction JS (ligne)	Appel API
Accueil	`home()` :3383	`/health`
Piéton	`pedestrian()` :3420	`/crossing/{id}`
Étudiant	`student()` :3478	`/eta`
Bus	`bus()` :3559	`/bus/{ligne}/status`
Ambulance	`ambulance()` :3618	`/ambulance/next-intersections`
Replay 2D / 3D	`replay()` :3659	`/replay/{run}?positions=true` + `/events`
Opérateur	`operator()` :3756	`/evaluation`, `/kpi` + lien Streamlit

Le replay interpole les positions enregistrées toutes les 5 s (`syncFleetFromReplay()` :1824), dessine les feux (`drawTrafficSignal()` :2426), les piétons (:2459), les bus (:2524) et l'ambulance (:2533) ; la 3D utilise three.js (`start3DSim()` :3249).

Tableau de bord Streamlit de l'opérateur (`app.py`)

8 onglets créés en `app.py:738-750` : Comparaison (:753), Tous les KPI (:892), Tests et paires (:949), Équité (:1056), **Sécurité et ambulance** (:1141), Courbes (:1283), Sensibilité des poids (:1350), Exports (:1378). Lancement : `python tasks.py operator` (port 8501).

> **Ne pas montrer** `run_mock.py` : c'est un serveur de maquette avec des données **inventées** (bandeau rouge « FAUSSES DONNÉES », port 8001). La démo officielle utilise l'API réelle sur le port 8000.

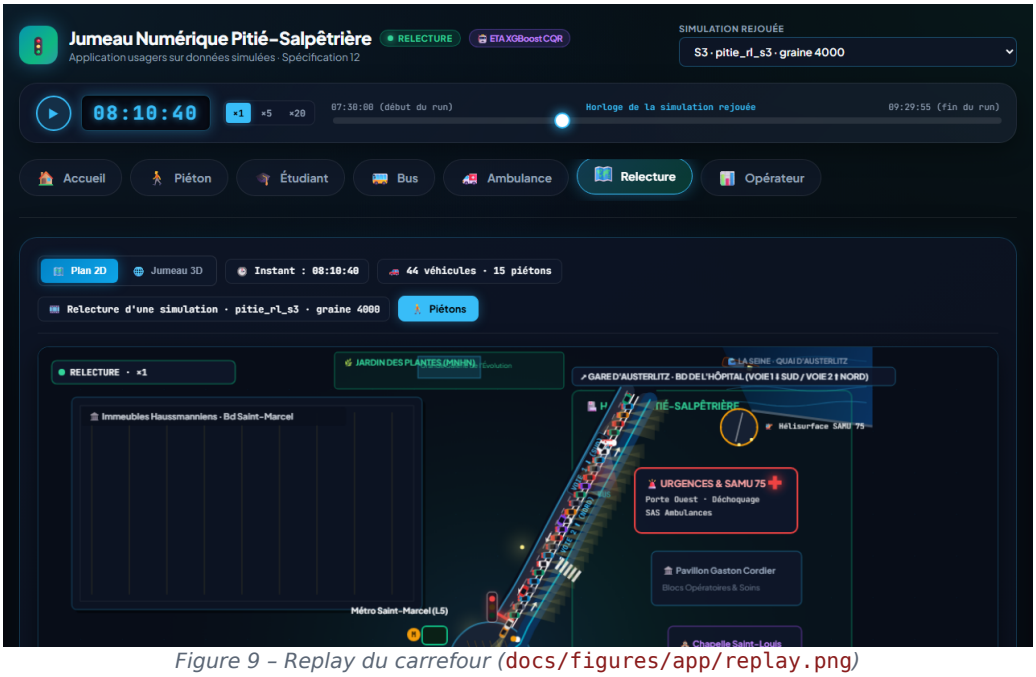


Figure 9 – Replay du carrefour (`docs/figures/app/replay.png`)

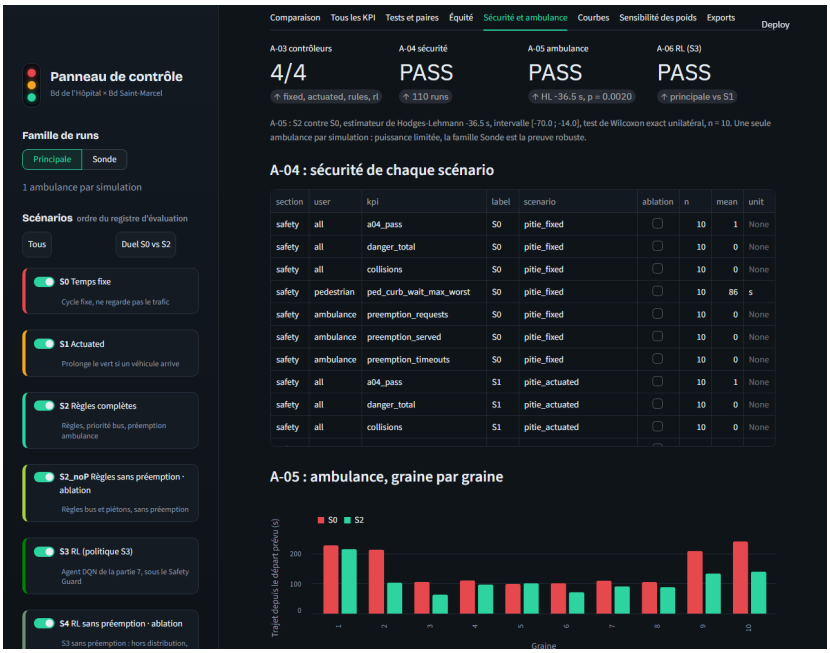


Figure 10 – Onglet « Sécurité et ambulance » de Streamlit (`docs/figures/app/streamlit_securite_et_ambulance.png`)

18. Lancer la démonstration (Windows)

> Sur cette machine, `data/` ne contient pas encore les résultats de démo : il faut lancer `bootstrap` une fois avant `demo`, sinon les vues seront vides.

Étape	Commande (PowerShell, à la racine du dépôt)
0. Aller dans le dépôt	`cd "C:\Users\hamdi\5DS6-9\Pi 5eme\urban-traffic-twin"`
1. (si besoin) installer	`.venv\Scripts\python.exe tasks.py install`
2. Restaurer les données de démo	`.venv\Scripts\python.exe tasks.py bootstrap`
3. Libérer les ports 8000/8501	`.venv\Scripts\python.exe tasks.py demo-clean`
4. Lancer API + Streamlit	`.venv\Scripts\python.exe tasks.py demo` (Ctrl+C pour arrêter)
Vérification rapide	`.venv\Scripts\python.exe tasks.py demo-check`

Parcours de démonstration en 8 étapes (graine 4000)

#	Vue	Adresse (préfixe `http://127.0.0.1:8000`)
1	Accueil	`/mock?view=home`
2	Piéton	`/mock?view=pedestrian&scenario=pitie_rules&seed=4000&t=1805&crossing=S&speed=normal`
3	Étudiant	`/mock?view=student&scenario=pitie_rules&seed=4000&t=2700&trajet=N_S&exam=08:18`
4	Bus (priorité 91 active)	`/mock?view=bus&scenario=pitie_rules&seed=4000&t=1305&line=91`
5	**Ambulance** (ONDE VERTE)	`/mock?view=ambulance&scenario=pitie_rules&seed=4000&t=2440&kiosk=1`
6	Replay (lecture ×5)	`/mock?view=replay&scenario=pitie_rl_s3&seed=4000&t=2440&play=0`
7	Opérateur	`/mock?view=operator`
8	Streamlit	`http://127.0.0.1:8501`

Source : `configs/demo.yaml:15-23` et `docs/demo/runbook.md`. Plan B si l'ordinateur tombe en panne : diapositives 12 et 15 de `docs/slides/urban_traffic_twin.pptx`.

19. Critères d'acceptation

ID	Critère	Statut	Preuve
A-01	Simulation sans interface et reproductible	PASS	70/70 runs refaits identiques à l'octet
A-02	Débit simulé proche du réel	PASS	6/6 lignes à ±15 % ; GEH < 5 sur 10/10
A-03	4 contrôleurs, même jeu de KPI	PASS	4/4, 83 KPI par famille
A-04	Aucune séquence dangereuse, piéton ≤ 90 s	PASS	110 runs, max 88 s
A-05	La préemption réduit le temps ambulance	PASS	HL −36,5 s, p = 0,0020
A-06	RL comparé aux références (10 graines)	PASS	S3 vs S1 : −9,5 s, p Holm 0,0059
A-07	Intervalle ETA 80 % couvre 75–85 %	PASS	80,6 % ; MAE 10,2 s vs 39,6 s
A-08	Toutes les vues passent par l'API	PASS	répétition enregistrée 2026-10-23
A-09	Démo sans intervention manuelle	PASS	clone neuf, 2026-10-23
A-10	Test avec 3 à 5 utilisateurs	**EN ATTENTE**	aucune session encore

Code qui calcule ce tableau : `src/traffic_twin/report/acceptance.py:83` ; tableau généré dans `docs/report/report.md`.

20. Les tests automatiques

Commande	Ce qu'elle fait
`.venv\Scripts\python.exe tasks.py check`	lint (ruff) + toute la suite pytest (ce que lance la CI GitHub)
`.venv\Scripts\python.exe -m pytest tests/test_safety.py -v`	règles du SafetyGuard
`.venv\Scripts\python.exe -m pytest tests/test_pitie_e2e.py -v`	simulations SUMO réelles de bout en bout (marqueur `sumo`)
`.venv\Scripts\python.exe -m pytest tests/test_guard_fuzz.py -v`	contrôleurs adverses aléatoires : la sécurité tient
`.venv\Scripts\python.exe -m pytest tests/test_evaluate.py -k a05 -v`	logique statistique du critère ambulance

Fichiers de test par domaine : simulation (`test_demand`, `test_signal`, `test_kpi`, `test_detectors`, `test_junction`, `test_incidents`), contrôle (`test_actuated`, `test_rules`, `test_preemption`, `test_pedestrian`, `test_safety`, `test_audit`), données (`test_clean`, `test_calendar`, `test_calibration_loop`, `test_flows`), ETA (`test_eta_*`), API (`test_api*`), application (`test_app_*`), évaluation (`test_evaluate*`, `test_compare`), rapport/démo (`test_report`, `test_demo`, `test_slides`).

21. Limites et questions probables du jury

Limites à assumer

- Programme de feux, parts de mouvements tournants, horaires de bus et débits piétons **supposés** (ASSUMED) en attendant la visite terrain.
- Validation des débits partiellement circulaire (demande construite sur les mêmes capteurs).
- Un seul carrefour modélisé : les deux autres carrefours du couloir ambulance sont « non modélisés ».
- Tests utilisateurs (A-10) et répétition humaine de la démo en attente.
- Le modèle ETA perd en couverture en cas d'incident (71,9 %).

Questions / réponses

Q. Où est l'ambulance dans le code ?

R. Création `sim/demand.py:191` (config `configs/pitie/demand.yaml:38`), détection `sim/emergency.py:95`, gestion `control/preemption.py:158`, application `control/safety.py:322` et `:367`, KPI `sim/kpi.py:284`, test statistique `evaluation/evaluate.py:1051`. Chapitre 7.

Q. Pourquoi la préemption est-elle dans le SafetyGuard et pas dans le contrôleur ?

R. Pour qu'elle s'applique de la même façon à **tous** les contrôleurs (y compris le RL) et qu'elle passe toujours par les vérifications de sécurité (vert min, dégagement, piétons).

Q. L'ambulance peut-elle mettre un piéton en danger ou le faire attendre trop ?

R. Non : chaque saut est vérifié par le bouclier piéton ; s'il ferait dépasser 90 s, la préemption cède (`ped_yield`, `control/safety.py:384-389`). Les dégagements ne sont jamais coupés (R1).

Q. Comment prouvez-vous que le gain n'est pas dû au hasard ?

R. 10 graines appariées, test de Wilcoxon exact pré-enregistré, effet Hodges-Lehmann avec IC 95 %, correction de Holm ; p = 0,0020 pour A-05.

Q. Pourquoi ne pas avoir choisi le RL s'il est meilleur ?

R. Règle pré-enregistrée d'équité : aucun usager ne doit être dégradé de plus de 5 % vs S1. Les piétons le sont de 5,2 % → FALLBACK sur S2. Décider la règle avant de voir les résultats évite de « choisir le gagnant » a posteriori.

Q. Que garantit la sécurité si le RL décide mal ?

R. Le RL ne fait que proposer « rester » ou « avancer » ; le SafetyGuard filtre, et l'audit indépendant recompte toutes les phases affichées (A-04 : 0 violation sur 110 runs).

Q. Comment la simulation est-elle reproductible ?

R. Graine SUMO fixée, config et demande hachées dans `run.json`, commit git enregistré ; A-01 : 70/70 runs refaits identiques à l'octet.

Q. D'où viennent les données réelles ?

R. Open Data Paris, capteurs permanents (nœud 302, arcs 508/511/513), `configs/data.yaml`.

Annexe A – Index rapide « je cherche… »

Je cherche…	Fichier : ligne (sous `src/traffic_twin/` sauf mention)
la boucle principale de simulation	`sim/run_scenario.py:222-228`
le lancement de SUMO	`sim/sumo.py:46` ; `sim/session.py:163`
un pas de simulation	`sim/session.py:277`
la génération des voitures	`sim/demand.py:247`
la génération des bus	`sim/demand.py:304`
la génération des piétons	`sim/demand.py:329`
la création de l'ambulance	`sim/demand.py:149` et `:191` ; `configs/pitie/demand.yaml:38`
les 7 ambulances (probe)	`configs/pitie/emergency_probe.yaml:9-16`
la détection de l'ambulance	`sim/emergency.py:95`
la préemption (FCFS, timeout)	`control/preemption.py:158`
le chemin de phases vers le vert ambulance	`control/preemption.py:48` `next_hop`
l'application de la préemption	`control/safety.py:322-325`, `:367`
le temps de l'ambulance (KPI)	`sim/kpi.py:284-294`
le test A-05	`evaluation/evaluate.py:1051`
l'ambulance dans l'interface	`app/mock.html:2533` (2D), `:3126` (3D), `:3618` (vue)
le statut ONDE VERTE (API)	`api/main.py:1378-1408`
la priorité bus	`sim/transit.py:30` ; `control/rules.py:73`, `:310-318`
le bouclier piéton	`control/pedestrian.py:108`, `:275` ; `control/safety.py:349`
le SafetyGuard	`control/safety.py:156`, `apply` `:310`
l'audit de sécurité	`sim/audit.py:65` ; A-04 `evaluation/evaluate.py:903`
le feu fixe S0	`control/fixed.py:29`
l'actuated S1	`control/actuated.py:72`
les règles S2	`control/rules.py:280`
le RL S3 (décision)	`control/rl.py:280`
l'environnement RL	`rl/env.py:347`
la récompense RL	`rl/reward.py:107`
le verdict FALLBACK	`rl/study.py:1732`
les KPI	`sim/kpi.py:136`, `:217`
le retard par personne	`evaluation/evaluate.py:746`
Wilcoxon / Hodges-Lehmann / Holm	`evaluation/stats.py:69`, `:215`, `:233`
le GEH	`evaluation/stats.py:245`
le nettoyage des capteurs	`data/clean.py`
le score du carrefour	`data/intersection_score.py:28`
la calibration	`data/calibrate.py:377` ; `validation/calibration_loop.py:62`
le modèle ETA	`ml/train_eta.py:339`, `:420`
le conseil de départ	`ml/advice.py:47`
les endpoints de l'API	`api/main.py:919-1219`
les onglets Streamlit	`app.py:738-750` (racine du dépôt)
le programme de feux SUMO	`sumo/network/pitie/pitie.tll.xml:51-69` (racine)
les paramètres de contrôle	`configs/pitie/control.yaml`, `configs/pitie/rules.yaml` (racine)
le protocole d'évaluation	`configs/evaluation.yaml` (racine)

Documents détaillés du dépôt : `README.md`, `docs/architecture.md`, `docs/evaluation.md`, `docs/validation.md`, `docs/eta.md`, `docs/rl.md`, `docs/rl_training.md`, `docs/api.md`, `docs/report/report.pdf` (rapport final en anglais), `docs/demo/runbook.md`.